Kishoreganj University

3rd Year 1st Semester B. Sc. (Engg.) Final Examination-2024

Department of Computer Science and Engineering

CSE 3105: Compiler Design (3 Credits)

Time: 3 Hours **Full Marks: 70**

Figures shown in the right margin indicate full marks.

Answer 05 out of 07 questions.

1 **a.**

digit	→	**[0-9]**
digits	→	**digits$^+$**
number	→	**digits (. digits)? (E[+-]? digits)?**
letter	→	**[A-Za-z]**
id	→	**letter (letter | digit)***
if	→	**if**
then	→	**then**
else	→	**else**
relop	→	**< | > | <= | >= | = | <>**

4

Consider the above regular expression, write down the tokens as <Token_Name, Attribute_Value> of the following C program:

```
int main(){
  // 2 variables
int a, b=2;
    a = 10;
 if(a==b){
     printf("%d is not larger", a);
 }
printf("Geeks are always smart, as they believe 5==5");
```

Omit the character sequences that are not matched with regular expression.

b. For the above regular expression in **1(a)**, draw transition diagram for **relop** token. 4

c. Summarize on the followings: 6

(i) Token
(ii) Patterns
(iii) Lexemes

2 **a.** Consider the following grammar: 6

S → D M Y | I Y | D J
I → D M
J → M Y
D → NN
M → October
Y → NNNN
N → 0|1|2|3|4|5|6|7|8|9

(i) Show the LMD and RMD for the string "**27 October 2025**" using the above grammar.
(ii) Explain how an ambiguous grammar can be determined. Identify whether the above-mentioned grammar is ambiguous or not.

b. Eliminate left-recursion from the following grammar:

K → KungfuPanda | Kungfu | Furious5 | $O_w$ogway

F → KFT | KFM | KFMa | KFV

O_w → KFmasterofmaster | FKshijumaster | $O_w$dragonKF | warriorKF

P → Po | ε

T → Tigress | ε

M → Monkey

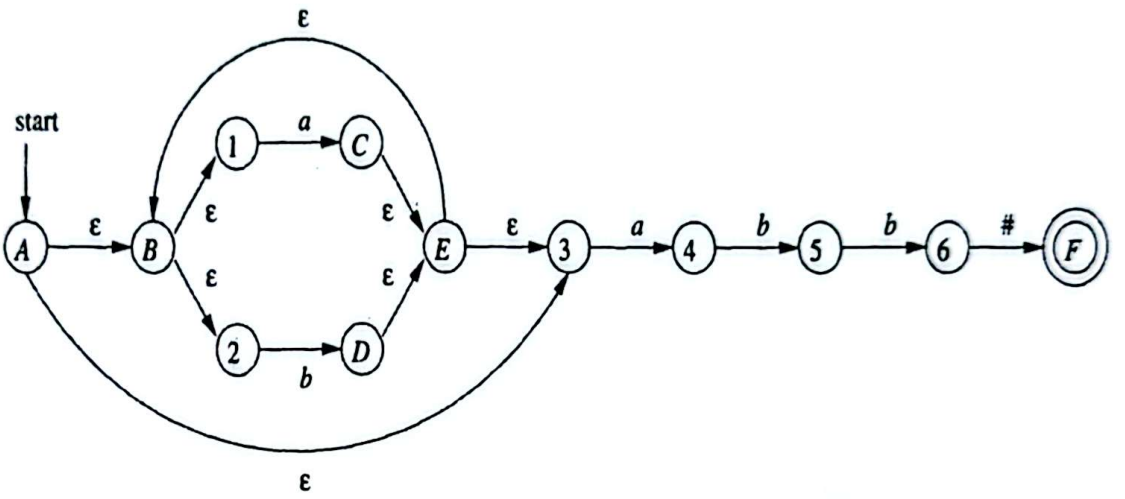
M_a → f$M_a$antis

V → fViper

C → fCrane

c. Write down the algorithm of elimination of left factoring. 3

3 a. Consider the following Finite Automata: 6



Give the Formal Definition of the above Finite Automata and identify whether it is a DFA or an NFA with proper explanations. If it is an NFA, then convert it into DFA using subset construction method.

b. Draw the NFA for the following expressions (Use Thompson's Construction Rules): 5

(i) $(((AB)^?C) \mid D)^+$

(ii) $AB^+DC \mid PQ^*R$

(iii) $MN^*T \mid XY^+$

(iv) $PR\ (P \mid R)^+\ PR$

(v) $(X^?|Y)^* \mid (YZ^*)^+$

c. Given a DFA missing several transitions, identify and explain how you would systematically locate and redirect all undefined transitions to maintain determinism. 3

4 a. Consider the following grammar to answer the following questions: 6

T → threeIdiotsTF| virus

F → farhanT | rajuT| ε

I → rancho

(i) Compute the FIRST and FOLLOW of all non-terminals in the given grammar.

(ii) Construct the corresponding LL parsing table.

(iii) State whether the given grammar is suitable to be parsed using the predictive parsing method or not. Use the previously constructed LL parsing table to give your answer. (Assume T is the starting state of the grammar)

b. Differentiate between Recursive Descent Parsing and Predictive Parsing. Consider the following grammar and generate the parse tree for the string **id*id+id** by employing Recursive Descent Parsing approach. 5

$$E \rightarrow T+E \mid T$$
$$T \rightarrow F*T \mid F$$
$$F \rightarrow (E) \mid id$$

c. Explain why LL(1) parsers require grammars to be free of left recursion and left factoring. 3

5 a. Identify whether there is shift-reduce conflict for LR(0) parsing for the following grammar by constructing the LR(0) parse table. 6

$$S \rightarrow Aa \mid bAc \mid dc \mid bda$$
$$A \rightarrow d$$

b. Analyze the approaches that can be adopted to resolve shift-reduce conflicts. Point out whether the SLR(1) parsing approach can resolve the shift-reduce conflict of the grammar of **question no 5(a)**. Construct the SLR(1) parse table for the grammar. 5

c. Construct the parse tree for the input string **abbcde** using the following grammar by employing Shift-Reduce parsing. 3

$$S \rightarrow aTRe$$
$$T \rightarrow Tbc \mid b$$
$$R \rightarrow d$$

6 a. Construct the 3-address code, Quadruple, Triple, and Indirect Triple for the following expressions: 6

(i) **x = (a+a)*b/c*((a+a+a+a)↑(c+d))*((a+a)-(a+a))**
(ii) **distance=starting_velocity*time+0.5*acceleration*time*time**

b. Differentiate between Parse Tree and Syntax Tree. Consider the following infix expressions: 5

(i) **((a↑b)+c)*(c-d)+((e-f)*(a/b))**
(ii) **a + (b - c) * ((d / e) ↑ f)**

Convert the given infix expressions into equivalent postfix expressions. Using the postfix expressions, construct the syntax trees by simulating the stack-based algorithm.

c. Show the constructing step of DAG of the following expression- 3

a + a * (b - c) + (b - c) * d

7 a. The following three address code is to implement the Quicksort Algorithm. Considering the following code answer the following questions. 6

1. i = m-1	11. t5 = a[t4]	21. a[t10] = x
2. j = n	12. if t5>v goto (9)	22. goto (5)
3. t1 = 4 * n	13. if i>=j goto (23)	23. t11 = 4 * i
4. v= a[t1]	14. t6 = 4 * i	24. x = a[t11]
5. i = i+1	15. x = a[t6]	25. t12 = 4 * i goto (19)
6. t2 = 4 * i goto (2)	16. t7 = 4 * i	26. x = a[t11]
7. t3 = a [t2]	17. t8 = 4 * j	27. t12 = 4 * I goto (15)
8. If t3<v goto (5)	18. t9 = a[t8]	28. a [t12] = t14
9. j = j -1	19. a[t7] = t9	29. t15 = 4 * m
10. t4 = 4 * j	20. t10 = 4 * j	30. a[t15] = x

(i) Identify the leaders for the code.

(ii) How many basic blocks are present in the above code?
(iii) Identify the instructions that are selected as leader more than 2 times.

b. Draw the Basic Block and Flow Graph for the above-mentioned code of **question no 7(a)**. 5

c. Identify the optimization techniques that can be applied on the following code. Rewrite the code after applying optimization techniques. 3

```
int main() {
    int a = 10, b = 2;
    int c = a + b;
    int f = a + b;
    int i = f;
    int temp2;
    while (i < 20) {
        temp2 = a * b;
        i++;
    }
    if (c < 0) {
        i = e + f;
    }
    return temp2 + i;
}
```